

GTX-GPU: Presenter Guide

Mid-term presentation (Part 1 of 2)

Everything the three presenters need to understand and explain their part

How to use this guide

Every presenter should read **Sections 1 and 2** (the glossary and the big picture), then their own section in depth, then the “one sentence” box of the other two parts, so that anyone can cover a teammate’s question.

Part	Presenter	Topic	Time
Opening	Presenter 1	What the project is, what today covers	1 min
Part 1	Presenter 1	The problem and our idea (§3)	4 min
Part 2	Presenter 2	How the design works (§4)	4 min
Part 3	Presenter 3	Correctness and first benchmarks (§5)	4 min
Close	Presenter 3	What comes next, questions	1 min

Scope of today. This is the *first half* of the project, told in the order the work was done: (1) understand the problem and the existing systems, (2) design the GPU data structure and its update rules, (3) build it, check that it is correct, and measure the engine on its own. The comparison against other systems, the stricter “serializable” correctness mode, and memory cleanup are the second half and are presented next time (§6).

Contents

1	Glossary: words you will hear and say	3
2	The big picture in one page	3
3	Presenter 1: The problem and our idea	5
3.1	Why graphs need transactions	5
3.2	Why GPUs, and what exists today	5
3.3	The GTX idea: never overwrite, add versions	6
3.4	Why a GPU needs changes	6
4	Presenter 2: How the design works	8
4.1	Idea 1: versions, not overwrites	8
4.2	Idea 2: growing storage without copying	8
4.3	Idea 3: coordinating without locks	9
5	Presenter 3: Correctness and first benchmarks	12
5.1	How readers stay consistent: batches (epochs)	12
5.2	How we know it is correct: the history checker	12
5.3	First benchmarks	13

6	What we are <i>not</i> presenting today	18
7	Team preparation checklist	18

1 Glossary: words you will hear and say

Term	Meaning, in plain words
Graph, vertex, edge	A network. Vertices are the things (users, accounts, road junctions); edges are the connections between them. An edge (u, v) goes from u to v .
Dynamic graph	A graph whose edges keep being inserted and deleted while people read it.
Hub vertex	A very popular vertex (a celebrity account) that a large share of all updates touch. Real graphs have these, and they are what make concurrency hard.
Transaction	A group of operations that must happen <i>all together or not at all</i> , e.g. “delete edge A and insert edge B”. Nobody may see only half of it.
Commit / abort	Commit = the transaction succeeded and its changes become visible. Abort = it clashed with another transaction, gives up, and retries.
Snapshot	A frozen, consistent view of the graph at one moment. A reader working on a snapshot is not disturbed by writers.
Snapshot isolation (SI)	The correctness guarantee we provide today: every transaction reads from a snapshot, and two transactions that change the same edge cannot both commit.
GPU thread, warp	A GPU runs thousands of threads. They are grouped in <i>warps</i> of 32 that execute the same instruction at the same time, in lockstep.
Atomic operation	A tiny operation the hardware guarantees nobody can interrupt halfway, e.g. “add 5 to this counter and tell me the old value” (<code>fetch_add</code>).
CAS	<i>Compare-and-swap</i> : “set this value to Y, but only if it is still X, which is what I saw a moment ago”. If someone changed it in between, the CAS fails and you look again. This is how threads cooperate without locks.
Lock	A “reserved” sign: only the holder may touch the data; everybody else waits. Locks are slow and risky on GPUs, which is why we avoid them.
Version / delta	One record of one change to one edge (“edge $u \rightarrow 7$ inserted by transaction 12”). We never overwrite data; we add versions.
Epoch	A batch number. Transactions commit in numbered batches (epochs).
Frontier	The newest epoch whose transactions are <i>all</i> finished. Readers take this number as their snapshot.
Throughput	How much work is done per second (transactions/s or edges/s).
Effective edges/s	Edge changes that actually changed the graph. Inserting an edge that already exists does nothing, so it is not counted.
Abort ratio	The fraction of attempts that had to give up and retry. Lower is better.
p50 / p99 latency	The time within which 50% / 99% of requests finish. p99 describes the slow tail.
GTX	A published CPU graph system (Zhou et al., SIGMOD 2025) whose design we adapt to the GPU.
GTX-GPU	Our system.

2 The big picture in one page

The problem. Graphs such as social networks, payment networks and road networks change all the time. Many changes only make sense in groups, and people keep reading the graph while it changes. GPUs are very good at updating graphs quickly, but none of the existing GPU graph systems can apply a *group* of changes safely while readers run.

The idea. A CPU system called GTX solves this problem on CPUs with one central trick: *never overwrite anything; store every change as a new version*. Readers look at the versions that existed when they started; writers add new versions next to them, so neither blocks the other. We take this design to the GPU.

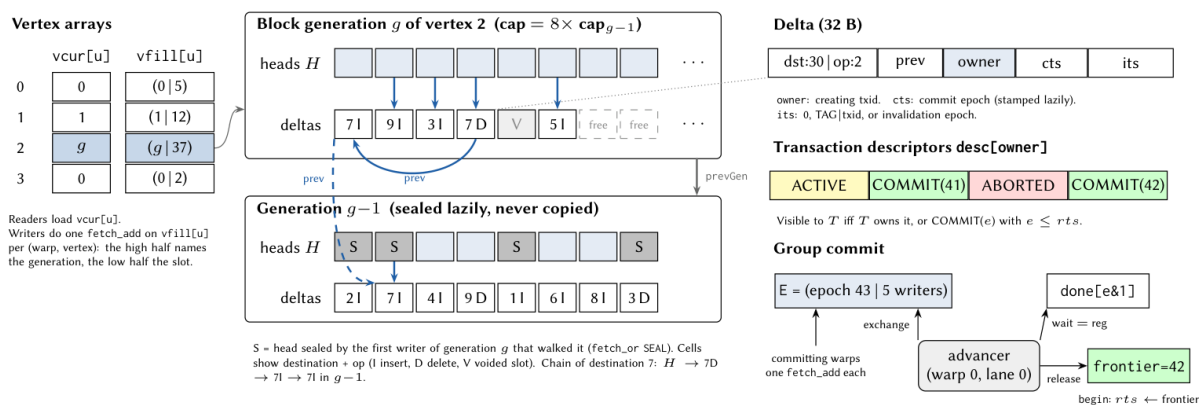


Figure 1: The whole system on one page (the same picture as Figure 1 of the technical report). Left: two small arrays per vertex. Middle: the vertex’s storage blocks, newest first, holding versions of its edges linked in chains. Right: the layout of one version, the status table of transactions, and the batch-commit machinery. Presenter 2 explains the left and middle; Presenter 3 explains the right.

3 Presenter 1: The problem and our idea

If you remember only one sentence

GPUs can update graphs very fast, but not safely in groups. We are bringing a proven CPU design, GTX, to the GPU to fix that.

3.1 Why graphs need transactions

Many real updates are not one edge but several edges that belong together:

- **Moving a connection.** A user switches from following account A to following account B: delete (user, A), insert (user, B).
- **Undirected edges.** A friendship is stored in both directions: insert (u, v) and (v, u).
- **Edge plus a counter.** Add an edge and increase the vertex's degree (its number of edges).

If other threads read the graph while such a group is half done, they see a state that never really existed. Figure 2 shows it: a reader that looks between the two steps sees the user following *nobody*. That reader might be a recommendation job or a fraud check, and it would compute a wrong answer.

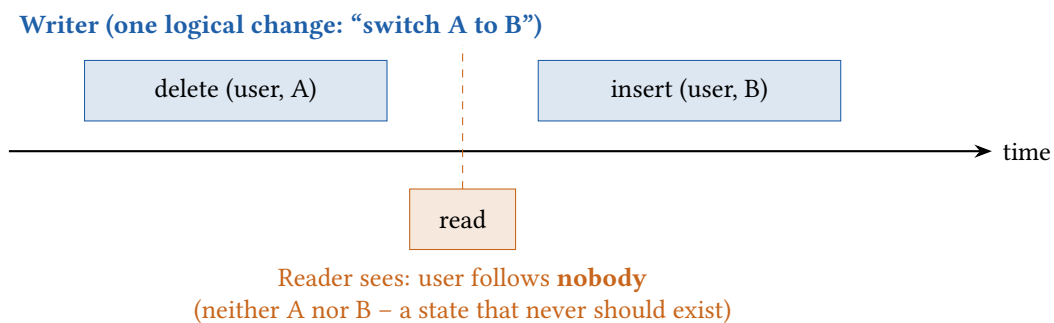


Figure 2: A “torn read”. Without transactions, a reader can observe half of a group of changes.

So we need two guarantees:

1. **All-or-nothing groups** (transactions): the two changes happen together or not at all.
2. **Consistent reading during updates** (snapshots): a reader sees the graph as it was at one moment, never half of a transaction.

3.2 Why GPUs, and what exists today

GPUs have thousands of threads and far more memory bandwidth than a CPU, and much graph analytics already runs on them. Several GPU structures for dynamic graphs exist, and they are fast: SlabGraph, for example, reports 500–640 million edge insertions per second. But they all work *batch at a time*: you hand them a big batch of updates, they apply it, and during that time nobody may read. None offers groups of changes that are all-or-nothing, or safe reading during updates.

System	How it stores edges	Transactions?	Safe reads during updates?
GPMA, GPMA+, LPMA	sorted arrays	no (batches)	no
Hornet, fainGraph	per-vertex arrays / pages	no	no
SlabGraph, Meerkat	per-vertex hash tables	no	no
GPU transactional memory (GPU-STM, PR-STM, CSMV)	any memory, not graph-aware	yes	yes, but slow on graphs
GTX (CPU only)	versioned edge records	yes	yes
GTX-GPU (ours)	versioned edge records	yes	yes

Why not just use GPU transactional memory? It treats memory as anonymous words and knows nothing about edges. Every insert into a popular vertex also updates that vertex’s edge counter, so thousands of unrelated transactions collide on that single counter and keep aborting each other. You only need to say this qualitatively today; the measurements belong to the final presentation.

3.3 The GTX idea: never overwrite, add versions

GTX (a SIGMOD 2025 CPU system) keeps, for every vertex, a list of *versions*. Each version records one change to one edge and *when* it happened. Its four ideas, which we keep:

1. **Versions instead of overwrites.** Readers look at old versions while writers add new ones, so readers never wait for writers.
2. **Per-edge chains.** The versions of one edge are linked together, so a writer can quickly check whether someone else is changing the same edge.
3. **Status lookup on demand.** A version knows which transaction wrote it; a reader checks that transaction’s status (still running, committed, aborted) only when it meets the version.
4. **Batch commits.** Transactions commit in numbered batches; a reader’s snapshot is simply “everything up to batch N ”.

GTX reports up to 11× higher throughput than earlier CPU graph systems (LiveGraph, Teseo, Sortledton).

Everyday analogy

A shared notebook where nobody ever erases. To change a phone number you write a new line with a timestamp. Someone reading “as of 10:00” simply ignores lines written after 10:00, so they never have to wait for the writer, and they never see a half-written entry.

3.4 Why a GPU needs changes

- **Lockstep groups of 32.** A warp’s 32 threads execute together and often target the same hub vertex. If each thread performs its own atomic operation on that vertex, they queue up one after another. A thread waiting for a lock inside a warp can even block the thread that holds it (deadlock).
- **No “stop the world”.** GTX copies a vertex’s storage into a bigger block when it is full. On a GPU, thousands of threads may be writing into that block at that instant, so it cannot be copied safely.
- **Bugs are silent.** GPU memory ordering is weak: if one “publish” step is missing, nothing crashes; a rare reader just sees wrong data. So we need a strong way to prove the system is correct (Presenter 3).

Our goal: keep GTX’s architecture, and redesign exactly the parts the GPU breaks.

What to say (script you can adapt)

“Graphs like social or payment networks change constantly, and many changes come in groups, like unfollowing one account and following another. If someone reads in the middle, they see a state that should never exist (Figure 2). So we need two things: all-or-nothing groups, and readers that always see a consistent picture.

GPUs are the fastest hardware for graph updates today, and there are several GPU graph systems, but they all apply updates in big batches with nobody reading. None of them supports safe groups of changes. General GPU transactional memory does, but it doesn’t understand graphs, so on popular vertices everything collides.

On CPUs, a system called GTX solved this with one idea: never overwrite, always add a new version. Readers look at older versions while writers add new ones. We are bringing GTX to the GPU. That’s not a simple port: GPU threads run in lockstep groups of 32, thousands of them hit the same popular vertex, and nothing can be paused to reorganize memory. [Presenter 2] will show how we redesigned it.”

Likely questions and simple answers

Why not do this on a CPU? GPUs offer far more parallelism and memory bandwidth, and graph analytics already runs on GPUs; moving the graph back and forth between CPU and GPU is expensive.

Is GTX your system? No. GTX is a published CPU system (Zhou et al., SIGMOD 2025). Ours is GTX-GPU: the GPU adaptation and its new mechanisms.

What guarantee do you give? Snapshot isolation today (defined in the glossary). A stricter mode (serializability) is part of the next stage.

Suggested slides for this part

1. Title and team; one-line pitch; “today = first half: problem, design, first results”.
2. “Graph updates come in groups”: the follow/unfollow example and Figure 2.
3. “No GPU system offers both”: the table above.
4. “GTX: never overwrite, add versions”: four ideas and the notebook analogy.
5. “Why the GPU needs changes”: three bullets, then the goal sentence.

4 Presenter 2: How the design works

If you remember only one sentence

We keep every version of every edge, we grow storage without copying it, and threads coordinate with single atomic operations and “check-then-swap” instead of locks.

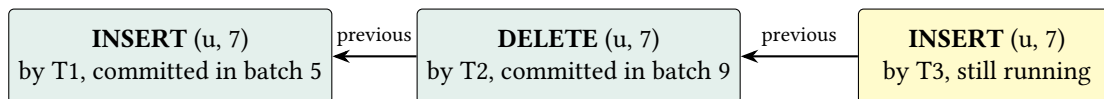
Use Figure 1 as your anchor slide. Point at three things only: the versions (middle), the storage blocks (middle, top and bottom), and the small per-vertex arrays (left).

4.1 Idea 1: versions, not overwrites

For every vertex we keep a list of small records (32 bytes each), one per change. Each record stores:

- **which edge and what happened:** the destination vertex and the operation (*insert* or *delete*);
- **who wrote it:** the id of the transaction that created it;
- **when it became official:** the commit batch number (filled in after commit);
- **a link to the previous version of the same edge**, which forms a chain per edge.

The current state of an edge, for a given reader, is simply the *newest version that reader is allowed to see*. A delete is not an erase; it is a new record saying “deleted”. Figure 3 shows the chain for one edge.



Reader with snapshot = batch 7 walks newest → oldest: skips T3 (not committed), skips T2 (batch 9 > 7), uses T1 ⇒ edge (u,7) is **present**. A reader with snapshot = batch 10 would see it **absent**.

Figure 3: Three versions of one edge. Different readers see different, but always consistent, states.

Two useful consequences:

- **Readers never wait.** They just skip versions that are too new.
- **Aborting is free.** If a transaction fails, its versions stay in memory but are ignored forever, because its status says “aborted”. Nothing needs to be undone.

4.2 Idea 2: growing storage without copying

Each vertex’s versions live in a storage block. When the block is full, GTX on the CPU copies everything into a bigger block. On the GPU, thousands of threads may be writing into that block at that exact moment, so we never copy:

1. The thread that asks for the first slot *beyond* the end is, by construction, the only one; it becomes the “grower”.
2. It creates a new block **8× larger** and links it *in front of* the old one (Figure 4).
3. It publishes the new block; other threads that were waiting simply retry and land in the new block.
4. The old block stays where it is and is still read. The first writer that later walks an edge’s chain into the old block marks (“seals”) that chain there, so from then on that edge’s new versions only go into the newest block. This keeps all versions of an edge in one clean order.

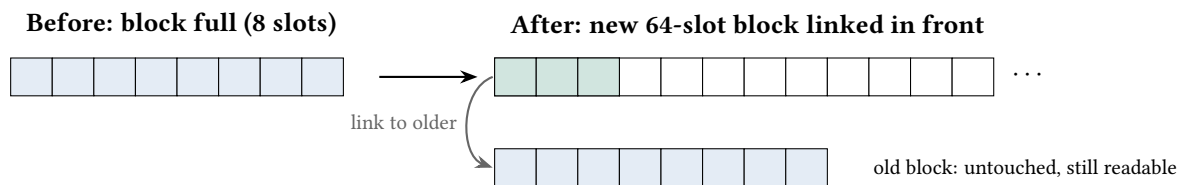


Figure 4: Growth by generations. Nothing is copied; new versions go into the new block (green).

Everyday analogy

When a notebook is full you do not rewrite it into a bigger notebook while ten people are writing in it. You put a new, thicker notebook on top of the old one and write there; the old notebook is still on the shelf if you need to look something up.

4.3 Idea 3: coordinating without locks

Getting a slot: one atomic per group, not per thread. Each vertex has a counter “how many slots are used” (vfill in Figure 1). Normally every thread that wants to write would increase it with its own atomic operation, and on a hub vertex 32 threads would queue on the same counter. Instead, the threads of a warp that target the same vertex first form a group (one GPU instruction, `__match_any_sync`); the group leader increases the counter *once* by the group size, and every thread takes “start + its position in the group” (Figure 5). A detail worth one sentence: the same 64-bit word stores both *which block* is current and *how many slots* are used, so one atomic answers both questions at once and a thread can never end up with a slot in the wrong block.

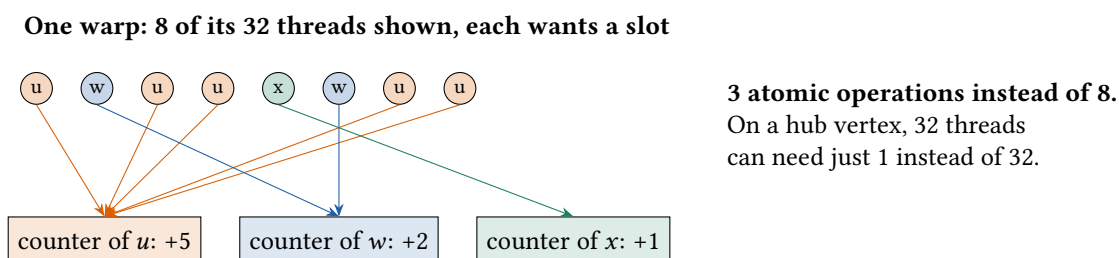


Figure 5: Warp-cooperative reservation. Threads that want space in the same vertex share one atomic operation.

Linking a new version: check, then swap. To insert or delete edge (u, v) , a thread:

1. reads the current start (“head”) of the chain for v ;
2. walks the versions of v in that chain: if another transaction that is still running has a version there, or a version was committed after its own snapshot, the two would conflict, so it **aborts** and retries later;
3. otherwise writes its version and links it with a **CAS on the same head it checked**: “make my version the head, but only if the head is still what I checked”;
4. if the CAS fails (someone linked something in between), it checks *only the new part*; if that part touches the same edge, it aborts, otherwise it simply tries again.

Figure 6 shows two threads racing for the same edge: exactly one wins, the other notices and aborts. No lock is ever held, so no thread waits for another.

Thread A (T5: delete (u,7))



→ time



Thread B (T6: delete (u,7))

Figure 6: Check-then-swap. Both threads checked the same head; only the first CAS succeeds. The loser re-checks only what changed, sees a conflicting version of the same edge, and aborts.

How this differs from GTX. GTX's rule is coarser: several edges share one chain, and GTX aborts if the newest version on the chain is not visible, *whichever edge it belongs to*. Two transactions touching *different* edges that happen to share a chain then abort each other for no reason. Our rule only looks at versions of the *same* edge. Presenter 3 shows what this is worth in numbers (Figure 9).

What to say (script you can adapt)

“Here is the whole system on one page [Figure 1]. Three ideas.

First, we never overwrite. Every change to an edge is a small new record: which edge, insert or delete, which transaction wrote it, and when it committed. The versions of one edge form a chain. A reader walks the chain from newest to oldest and takes the first version it is allowed to see [Figure 3]. So readers never wait, and if a transaction fails, there is nothing to undo; its records are simply ignored. Second, storage grows without copying. When a vertex's block is full, exactly one thread creates a block eight times bigger and links it in front. The old block stays readable. On a GPU that matters, because thousands of threads may be writing into that block at that moment [Figure 4].

Third, no locks. Threads in a warp that want space in the same vertex share one atomic operation instead of each doing their own [Figure 5]. To add a version, a thread checks the chain of that exact edge and then links its version with compare-and-swap, ‘only if nothing changed since I looked’. If two threads race for the same edge, exactly one wins and the other aborts and retries [Figure 6]. GTX checks conflicts more coarsely, which causes unnecessary aborts; [Presenter 3] will show the difference, and how we know all this is correct.”

Likely questions and simple answers

Doesn't keeping every version use a lot of memory? Yes. Cleaning up versions nobody can see any more is planned for the next stage.

Why 8× bigger blocks? Growing by a large factor means growth happens rarely, so the (rare) waiting for the one growing thread is negligible.

What if two threads want the same slot? They can't: one atomic operation hands out distinct slot numbers.

Is compare-and-swap a lock? No. Nobody holds anything; a failed CAS just means “look again”. Someone else's CAS succeeded, so the system as a whole always makes progress.

Suggested slides for this part

1. “The whole system on one page”: Figure 1.
2. “Every change is a new version”: Figure 3.
3. “Storage grows without copying”: Figure 4 and the notebook analogy.
4. “One atomic per group”: Figure 5.

5. “Check, then swap”: Figure 6, plus the one-line contrast with GTX.

5 Presenter 3: Correctness and first benchmarks

If you remember only one sentence

Readers see consistent batches and never wait; an automatic checker verifies every single read and everything passes; and the engine already processes over 100 million edge updates per second, up to 1.8× faster than GTX’s original rules.

5.1 How readers stay consistent: batches (epochs)

Transactions commit in numbered batches called *epochs*. The mechanism has three players (right-hand side of Figure 1):

- **Committing transactions** join the currently open batch. A whole warp joins with *one* atomic operation on a single counter that holds both “current batch number” and “how many writers joined”. The transaction then marks itself “committed in batch *e*” and reports “done”.
- **One special thread, the advancer**, closes the open batch (opening the next one), waits until every writer that joined the closed batch has reported done, and only then announces: “batch *e* is complete”, by updating a single number called the **frontier**.
- **A reader** starts by reading the frontier; that number is its snapshot. It then sees exactly the versions of batches up to the frontier (Figure 3).

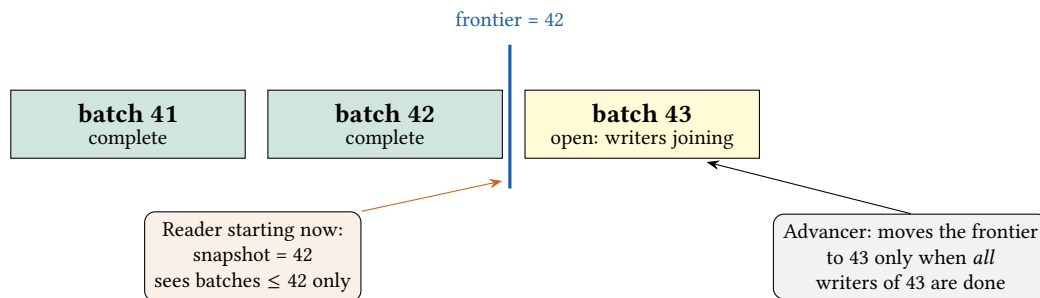


Figure 7: Batch commit. Because the frontier only moves past a batch when every transaction in it is finished, a reader can never see half a transaction, and it never has to wait.

Why this matters: a transaction’s versions all carry the same batch number, and readers only see complete batches. So a reader sees either *all* of a transaction or *none* of it, with no waiting and no locks.

A story worth telling (it shows real engineering). Our first version of “join the batch” read the batch number, registered, then read it again to make sure it had not changed, and retried if it had. The advancer changes the batch number about once per microsecond, so under load a committing thread kept retrying and a run that should take 1 ms sometimes took 1.5–20 seconds (a *livelock*). Packing the batch number and the writer count into one word, so that joining is a single atomic operation that always succeeds, fixed it: 180 of 180 repeated runs then finished in 0.15–1.0 ms.

5.2 How we know it is correct: the history checker

GPU concurrency bugs rarely crash; they produce a *wrong read* once in a million operations. The usual test in GPU prototypes (“does the final graph equal the list of logged changes?”) cannot see such bugs. Example: two transactions both delete the same edge and both report success. The final graph looks fine (the edge is gone), but one of them should have aborted.

So every run records, for every transaction, its snapshot number, its commit batch, and the result of *every* operation. A checker on the CPU then replays the history and verifies:

1. **Every read is right:** each lookup and scan returned exactly what the graph looked like at that transaction’s snapshot (plus its own earlier changes).
2. **The right version was read:** a lookup names the exact version it saw, never one from an aborted or unfinished transaction.
3. **No lost updates:** two transactions that changed the same edge never overlap in time.
4. **The final graph is right.**
5. A fifth check (no ordering cycles) is used for the stricter mode planned for the next stage.

Results.

- **48 of 48** adversarial test histories pass: six nasty workloads (tiny graphs with heavy collisions, a hub vertex taking 50% of operations, 8 edges everyone fights over, delete/re-insert churn, large transactions, many readers) $\times$ 8 configurations.
- **The checker catches bugs:** we deliberately broke the system in three ways (no conflict check, readers ignore their snapshot, readers see unfinished data). The checker caught all three, with thousands of violations each. Two of the three *pass* the simple final-graph test, which is exactly why we built the checker.
- Running out of memory is handled safely: the transactions that did commit are still consistent.

5.3 First benchmarks

Setup (say this once). Laptop GPU: NVIDIA RTX 4060 (8 GB), CUDA 12.8. Graph with $2^{20} \approx 1$ million vertices and 4 million starting edges; each run processes about 1 million operations. Each number is the median of 5 runs after a warm-up run.

Workload names. *Uniform:* edges between random vertices, so few collisions. *50% hub:* half of all operations touch one single vertex, a worst case for collisions. *Zipf* and *R-MAT:* realistic “few very popular, many unpopular” graphs. K = the number of edge operations per transaction. *GTX rules* = our engine run with GTX’s original, coarser conflict rule and per-thread reservation, i.e. a direct port of GTX.

Benchmark 1: speed as transactions grow

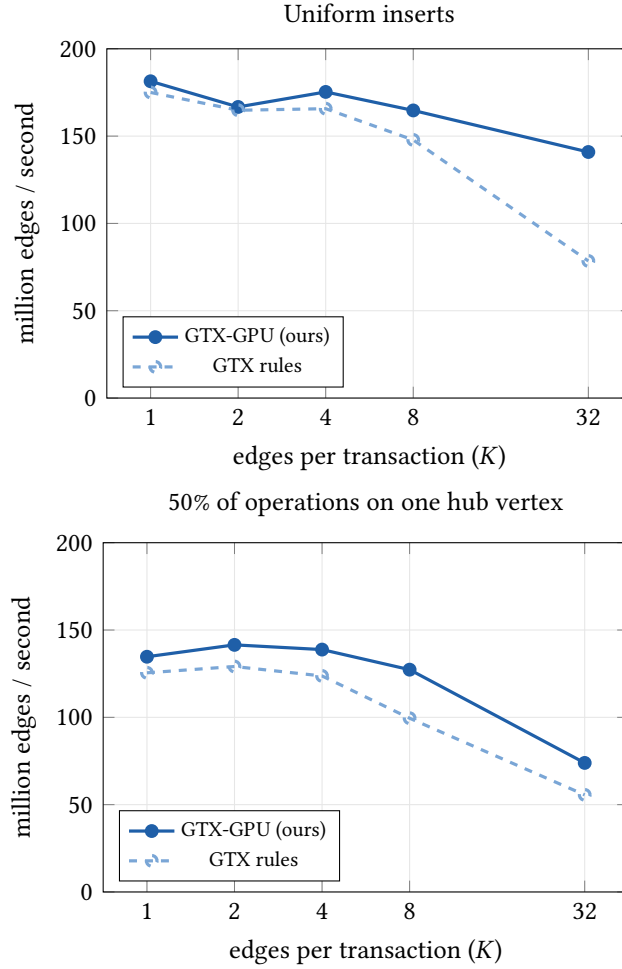


Figure 8: Effective edge insertions per second. Our engine stays at 140–180 million/s on uniform graphs for every transaction size, and above 120 million/s up to $K = 8$ even when half of all operations hit one vertex.

Million effective edges/s, by K	1	2	4	8	32
Uniform, GTX-GPU	181.4	166.7	175.3	164.7	140.9
Uniform, GTX rules	175.0	164.8	165.7	147.8	78.5
50% hub, GTX-GPU	134.7	141.5	138.8	127.3	73.9
50% hub, GTX rules	125.5	129.1	123.7	99.5	55.6

How to read it: flat lines are good; they mean bigger transactions do not make the system slower per edge. Ours stays flat; GTX’s rules drop sharply at $K = 32$. On the hub workload even ours drops at $K = 32$, because 32-edge transactions that all touch the same vertex genuinely collide (these are real conflicts, not false ones).

Benchmark 2: our conflict rule removes unnecessary aborts

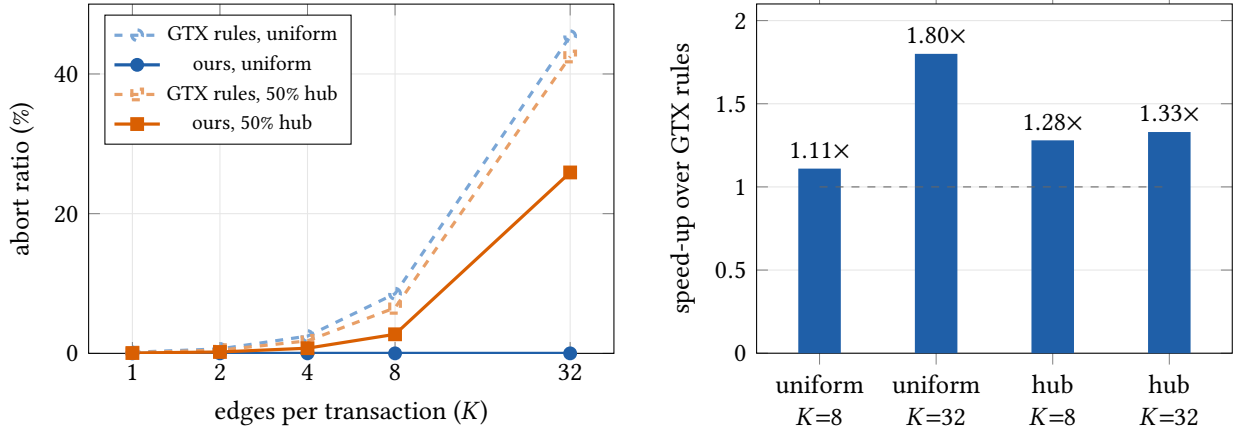


Figure 9: Left: fraction of attempts that abort. With GTX’s rule, aborts grow with transaction size to 45%; with ours they are zero on uniform graphs, and on the hub the remaining 26% are true conflicts on the same edges. Right: the resulting throughput gain.

How to read it: an abort is wasted work. With GTX’s rule, transactions that touch *different* edges still abort each other when those edges share a chain, and the bigger the transaction, the more likely that is. At 32 edges per transaction almost half of all attempts are thrown away. Checking conflicts per edge removes all of those false aborts, which makes the system 1.8× faster at that size. At $K = 1$ the two are within a few percent, as expected, because false conflicts are rare for single-edge transactions.

Benchmark 3: realistic mixes and concurrent readers

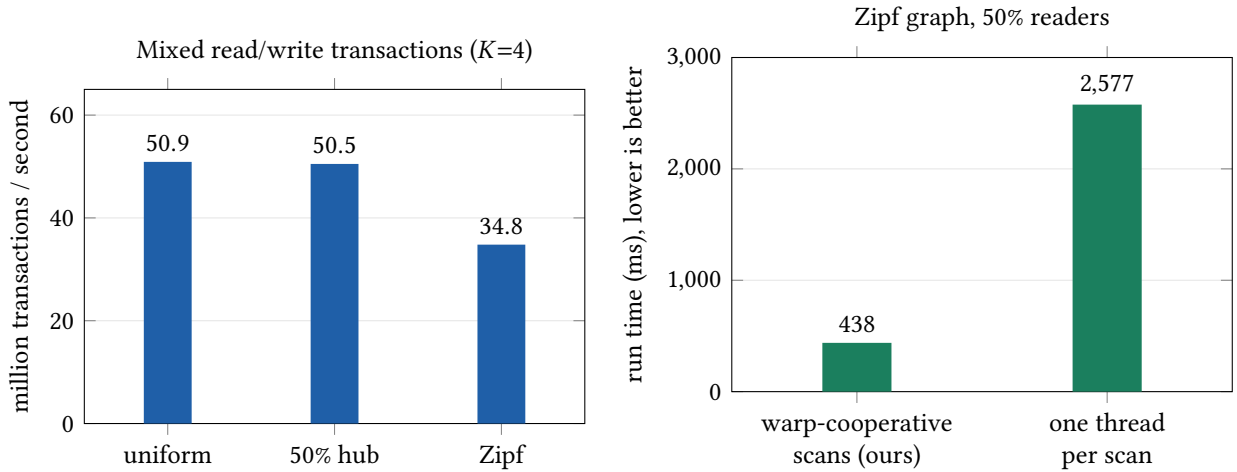


Figure 10: Left: committed transactions per second for a mix of 40% inserts, 30% deletes and 30% reads. Right: letting all 32 threads of a warp scan one vertex together is 5.9× faster than one thread per scan.

Mixed workload ($K = 4$)	transactions/s	effective edges/s	abort ratio
Uniform	50.9 M	140 M	1.1%
50% hub	50.5 M	105 M	1.9%
Zipf	34.8 M	94 M	1.1%

Readers. On an R-MAT graph where **90%** of transactions are read-only (10% of their operations scan a whole vertex's neighbour list):

- **17.3 million** transactions per second;
- overall abort ratio **0.03%**;
- read-only transactions: half finish within **0.64 ms** (p50), 99% within **2.4 ms** (p99), including scans. Readers never wait and never abort.

Other supporting numbers (use if asked): delete/insert churn with single-edge transactions runs at 156 million effective edges/s (uniform) and 144 million/s (R-MAT); the batch-commit machinery costs only 9.6–15.5% of the GPU's time for single-edge inserts.

What to say (script you can adapt)

“How do readers stay consistent? Transactions commit in numbered batches. A whole warp joins a batch with a single atomic operation. One special thread closes each batch and, only when every writer in it is done, announces it as complete by moving a number we call the frontier. A reader just reads the frontier and ignores anything newer [Figure 7]. So a reader sees all of a transaction or none of it, and never waits. Our first design retried when the batch number changed, and since it changes every microsecond, it could spin for seconds; making it one atomic operation fixed that.

How do we know it's correct? GPU concurrency bugs don't crash; they give a wrong answer once in a million. So we record every operation's result and a checker replays the whole history and verifies every single read. All 48 adversarial scenarios pass. We also broke the system on purpose three ways; the checker caught all three, and two of them would have passed the usual 'compare the final graph' test.

First numbers, on a laptop RTX 4060 with a million-vertex graph: 140 to 180 million edge inserts per second, steady as transactions grow [Figure 8]. Our per-edge conflict rule removes the unnecessary aborts of GTX's rule: at 32 edges per transaction GTX's rule throws away 45% of attempts, ours none, which makes us 1.8 times faster [Figure 9]. With 90% readers we run 17 million transactions per second and 99% of reads finish within 2.4 milliseconds.

Next stage: a stricter correctness mode, the full comparison against other GPU systems, and cleaning up old versions. Thank you, questions?”

Likely questions and simple answers

How does it compare to other systems? That is the next stage: the competing approaches (locking and GPU transactional memory) are implemented and the full comparison is the next milestone.

Why is the hub workload slower at $K = 32$? 32-edge transactions that all touch the same vertex genuinely conflict; those 26% aborts are real conflicts, not false ones. GTX's rule adds 17 more percentage points of false ones.

Why is “effective edges/s” lower than transactions/s on the hub? Many hub inserts try to add an edge that already exists; that is a no-op and is not counted as an effective change.

Why a laptop GPU? It is the hardware we have. All systems in the full comparison run on the same machine, so comparisons are fair; absolute numbers would be higher on a data-centre GPU.

What is the advancer? Isn't one thread a bottleneck? It only moves a counter forward; commit machinery is 9.6–15.5% of GPU time in the worst (single-edge) case.

Suggested slides for this part

1. “Readers see complete batches”: Figure 7, plus the livelock story in one sentence.
2. “We check every read”: the list of checks and the results (48/48, 3/3 bugs caught).

3. “Speed as transactions grow”: Figure 8.
4. “Fewer wasted attempts”: Figure 9.
5. “Readers never wait”: the reader numbers and Figure 10 (right).
6. “Next stage” and questions.

6 What we are *not* presenting today

Keep this for the final presentation

These exist and are done, but they are the second half of the project. Do not bring them up; if asked, say “that is our next milestone”.

- **Comparison against other systems** (two-phase locking, TL2 GPU transactional memory, non-transactional updates, GPMA+) and the large speedups from it.
- **Serializable mode** (the stricter correctness level) and the write-skew test.
- **Hypothesis tests / ablations**, including the finding that one-atomic-per-warp reservation helps only a little (3–11%).
- **Where we are slower** (uncontended small transactions versus locking) and the **memory overhead** (2.5–20×), which cleanup of old versions will address.
- The list of protocol bugs found and fixed (except the livelock story above), memory-ordering details and formal invariants.

7 Team preparation checklist

- Everyone: read Sections 1 and 2, your own section, and the other two “one sentence” boxes.
- Presenter 1: be able to explain the torn-read example and why GPU transactional memory struggles with hub vertices.
- Presenter 2: be able to walk Figure 3 (which version a reader sees) and Figure 6 (who wins a race) without notes.
- Presenter 3: be able to explain why a reader can never see half a transaction (Figure 7) and how to read Figures 8 and 9.
- Rehearse the two handoff sentences so the talk flows as one story.

Where the numbers come from (in case someone asks to see the data): `results/summary.csv`, `results/speedups.csv`, `results/correctness_suite.txt` and `results/mutation_final_state.txt` in the project repository; the full technical report is `paper/GTX-GPU_Technical_Report.pdf`.